

PROJETO ARANDU

Treinador de Depuração

Arquitetura, fluxo de informação e plano de implementação

AKCIT Camp Multicêntrico 2026 — Trilha Norte (Manaus)

Documento de arquitetura da equipe · versão 1 · 23/09/2026

Para que serve este documento: que os quatro integrantes entendam, de forma clara, a mesma coisa: o que é a ideia, como as peças conversam entre si, quem constrói o quê, e em que formato cada peça entrega o seu resultado para as outras. Sempre que aparecer um termo técnico, ele é explicado na primeira vez que surge (e há um glossário na seção 2).

1. Visão geral

O problema

Nas primeiras disciplinas de programação, o juiz online (o sistema que corrige os exercícios automaticamente) só responde “certo” ou “errado”. Quem erra não descobre o que entendeu de forma equivocada e fica tentando no escuro até desistir. O professor, com turma grande e poucos monitores, não consegue acompanhar cada aluno. E depurar — encontrar e corrigir o próprio erro — é a habilidade que mais separa quem avança de quem trava, mas quase nunca é treinada de propósito.

Isso tem tamanho: o INEP aponta cerca de 23% de reprovação e evasão nas disciplinas de lógica de programação, e um estudo da UFPR mostrou que 79% dos alunos reprovados em Algoritmos e Programação abandonaram o curso. No interior do Amazonas, onde faltam professores e monitores, o problema tende a ser ainda maior.

A ideia, em três frases

1. O Arandu é um treinador de depuração que funciona sozinho, fora de qualquer plataforma. O aluno recebe um programa curto que contém um erro realista, roda as entradas que quiser, acompanha o valor das variáveis passo a passo, conserta o código e explica o erro em uma frase.
2. Por trás, um agente de IA observa como o aluno depura, monta um retrato do que ele já domina, e decide a cada momento o próximo passo: dar uma dica, mostrar os valores reais da execução, repetir o mesmo tipo de erro em outro contexto, avançar ou encerrar.
3. Tudo que o agente afirma sobre o código é provado rodando o programa. Ele não “acha”: ele executa e verifica.

Por que é diferente (o “moat”)

Moat, em inglês, é o “fosso” que protegia um castelo; no mundo de startups virou o termo para *a vantagem difícil de copiar*. A nossa não é o catálogo de erros — isso qualquer um monta. São três coisas combinadas:

- **O retrato individual de cada aluno** (o que ele domina e como depura), que fica melhor quanto mais ele usa o sistema.
- **Os dados de longo prazo** que ligam “que erro o aluno cometeu” a “ele continuou ou desistiu do curso” — algo que só se obtém com tempo e acesso a uma base real.
- **A verificação por execução**: o agente nunca fala do código sem antes rodar. Isso elimina o problema que derruba os tutores de IA atuais, que inventam o comportamento do programa.

2. Glossário rápido

Os termos abaixo aparecem no documento inteiro. Não precisa decorar; use como consulta.

Termo	O que é, em uma frase
Refactory	Uma base pública de código de alunos iniciantes em Python — com versões erradas, versões corretas e os casos de teste. É de onde tiramos os erros reais no protótipo.
ProgMiscon	Um catálogo pronto de “equivocos” de programação (crenças erradas comuns). Tem 31 fichas para Python. Usamos os nomes dele para rotular cada erro.
Equívoco (misconception)	A ideia errada na cabeça do aluno (ex.: “o if repete como um while”), que é a causa do bug. Trabalhamos no nível do equívoco, não da linha.
Bug / erro	O defeito no código. Aqui foco em erros de lógica: o programa roda, não trava, mas dá resposta errada.
Trace (rastros de execução)	A gravação do valor de cada variável a cada passo em que o programa roda. É o que deixa o aluno “ver por dentro”.
diff comportamental	Rodar o código do aluno e um código correto lado a lado com a mesma entrada e achar o primeiro ponto em que os dois divergem. É como localizar o erro de lógica sem chute.
Sandbox	Um ambiente isolado onde o código do aluno roda sem risco para o resto do sistema.
MCP	Um padrão para expor “ferramentas” que o agente pode acionar — pense num cardápio de funções que ele chama quando precisa (rodar código, pegar o trace, etc.).
RAG	Buscar informação relevante e entregá-la ao agente antes de ele decidir. Aqui: buscar como outros alunos saíram do mesmo erro.
Modelo do aluno	O retrato guardado de cada aluno: o que ele domina (por equívoco) e como ele depura.
Guardrail	Uma trava de segurança que impede uma saída ruim de chegar ao aluno (ex.: uma mensagem que entregaria a resposta).
Cold start / flywheel	Cold start = o começo, quando ainda não há usuários e faltam dados. Flywheel (“volante”) = o efeito de melhorar sozinho conforme mais gente usa.
Ablação	Testar o sistema com e sem uma peça para provar que aquela peça faz diferença.

3. Como a solução funciona

O sistema tem duas fases. A Fase A monta o material (roda uma vez, nos bastidores). A Fase B é a sessão com o aluno (roda toda vez que alguém usa).

Fase A — montar o banco de bugs (offline, roda uma vez)

O objetivo é ter, no fim, um conjunto de bugs bons: cada um é um programinha com um erro de lógica realista, acompanhado do código correto, dos testes, do nome do equívoco e de uma nota de dificuldade.

1. **Pegar os dados.** Partimos do Refactory: código de alunos reais, com versões erradas, a correta e os testes.
2. **Rotular cada erro.** Damos um nome ao equívoco por trás do bug, usando o catálogo ProgMiscon (ex.: “confundi if com while”). Assim o sistema fala a mesma língua em todo lugar.
3. **Construtor gera um erro novo.** Um agente reproduz aquele mesmo equívoco em um programa diferente. Fazemos isso para ter variedade e para não expor o código real de nenhum aluno.
4. **Validador confere rodando.** Outro agente executa o programa gerado: ele tem que falhar só onde deveria, não pode travar, e um conserto pequeno tem que resolver. Se reprovar, volta para o Construtor com o motivo. Esse vai-e-volta que só termina quando passa é o “laço” de verdade — o agente age, verifica e corrige.
5. **Calibrar a dificuldade.** A dificuldade vem do número de tentativas que alunos reais precisaram diante daquele equívoco — não do que a IA acha difícil.
6. **Minerar trajetórias.** De cada par (errado → certo) extraímos um “caminho de conserto”. Esse conjunto vira o material que o RAG consulta depois.

Resultado da Fase A: o Banco de Bugs (o que ensinar) e a Loja de Trajetórias (como os alunos costumam se desenrolar).

Fase B — a sessão com o aluno (roda sempre)

Aqui entram três agentes com papéis distintos e três lugares de memória. Um agente é um programa de IA que decide e age (não só responde texto).

Os três agentes

- **Construtor.** Cria e valida os bugs (é o motor da Fase A; na Fase B ele é chamado quando o Treinador quer repetir um equívoco em outro contexto).
- **Treinador.** O cérebro. Conduz a sessão: olha o modelo do aluno, escolhe o próximo bug, decide quando e como intervir (dica, mostrar valores, repetir, avançar, encerrar) e atualiza o retrato do aluno.
- **Verificador.** O guardrail. Antes de qualquer mensagem chegar ao aluno, ele confere duas coisas: (a) a mensagem está apoiada numa execução real? (b) ela não entrega a solução pronta?

A sub-rotina investigativa (dentro do Treinador)

Quando o Treinador decide dar uma dica boa, ele não chuta. Ele levanta duas hipóteses do que o aluno entendeu errado, fabrica uma entrada que faça as duas hipóteses darem resultados diferentes, roda o código e observa qual hipótese sobrevive. Assim a dica sai ancorada em valores reais. (Isso já existe na

literatura — um trabalho chamado LDB faz exatamente esse tipo de verificação passo a passo —, então não é aposta arriscada.)

Os três lugares de memória

- **Banco de bugs** — compartilhado por todos.
- **Sessão atual** — o que está acontecendo agora (curto prazo).
- **Modelo do aluno** — o retrato que persiste entre sessões (longo prazo).

4. O que vamos usar e por quê

Cada peça abaixo tem uma linha de “o que é” e uma de “por que usamos”. A regra geral do projeto é: usar o método determinístico (uma conta ou uma execução que sempre dá o mesmo resultado) onde ele resolve, e o modelo de IA só onde é realmente necessário.

Refactory (a fonte de dados no protótipo)

- **O que é:** base pública com ~2.400 códigos corretos e ~1.780 com bug, de 361 alunos, em 5 exercícios de Python iniciante, com casos de teste.
- **Por que usamos:** tem tudo que precisamos para o protótipo (erro + correto + testes) e é pública. A base do CodeBench (do IComp/UFAM) fica como fonte da fase final, quando tivermos um recorte anonimizado.

ProgMiscon (os rótulos de equívoco)

- **O que é:** catálogo pronto de equívocos de programação, com 31 fichas de Python (ex.: IfIsLoop, ConditionallsSequence).
- **Por que usamos:** dá um vocabulário com respaldo para nomear os erros, em vez de inventarmos categorias. Onde uma ficha não encaixa (alguns erros de lógica não têm ficha), criamos 2–3 rótulos próprios e declaramos isso.

Importante: o ProgMiscon **não** é uma base de códigos com bug — é um catálogo de descrições. Os bugs vêm do Refactory; o ProgMiscon só fornece o nome do equívoco.

Ferramentas de execução via MCP

- **O que é:** um conjunto de funções — rodar código, pegar o trace, rodar a suíte de testes, o diff comportamental, gerar uma entrada que separa hipóteses — expostas como um “cardápio” (MCP) que o agente aciona.
- **Por que usamos:** separa o que é determinístico (a execução) do que é decisão de IA (o Treinador). É o que prova, na prática, que o agente age no mundo, não só conversa.

RAG (aprender com quem já passou pelo mesmo erro)

- **O que é:** antes de decidir, o Treinador busca casos parecidos e usa isso para escolher melhor.

- **De onde vêm os dados:** no protótipo, das trajetórias mineradas do Refactory (Fonte A). Em produção, do próprio uso do sistema (Fonte B): cada sessão registra “perfil do aluno → dica dada → destravou ou não”.
- **Por que usamos:** no começo não há usuários (isso é o cold start), então começamos com a Fonte A. Conforme o sistema é usado, a Fonte B cresce e o RAG fica melhor sozinho — é o flywheel, e é parte do moat.

Verificador / guardrail

- **O que é:** a trava que checa toda mensagem antes de ela chegar ao aluno.
- **Por que usamos:** os tutores de IA de hoje pedem “por favor, não conte a resposta” no texto — e vazam mesmo assim. Nós verificamos por execução se a mensagem permitiria copiar e passar. É mais difícil de burlar.

Modelo do aluno

- **O que é:** por equívoco, uma estimativa de quanto o aluno domina aquilo, atualizada a cada ação (um método clássico chamado BKT faz isso: estima a chance de o aluno já dominar um conceito e ajusta a cada resposta). Mais os sinais de como ele depura.
- **Por que usamos:** é o que permite personalizar de verdade e é o diferencial que uma coordenação de curso compra (o mapa de equívocos da turma).

5. O fluxo de informação

Esta seção mostra como as peças conversam. Primeiro, uma sessão real contada passo a passo; depois, uma tabela com o dado que cada seta carrega.

Uma sessão, do início ao fim (exemplo)

A aluna Marina abre o Arandu. Acompanhe o que cada peça faz:

- 1. Escolha do bug.** O Treinador consulta o Modelo da Marina (ela ainda erra “condição de parada”) e, via RAG, vê que para esse equívoco mostrar os valores das variáveis costuma destravar alunos parecidos. Pede ao Banco de Bugs um bug de laço da dificuldade certa.
- 2. Investigação do aluno.** Marina lê o programa e roda uma entrada. A Interface chama a ferramenta executar e mostra a saída. Ela pede para ver passo a passo; a ferramenta trace devolve o valor de cada variável em cada linha.
- 3. Primeira tentativa.** Ela conserta o que acha que é o erro e roda os testes (ferramenta rodar_suite): passa em 4 de 5 e trava.
- 4. Pedido de dica → sub-rotina investigativa.** O Treinador decide intervir. Levanta duas hipóteses do equívoco, pede à ferramenta gerar_entrada_discriminante uma entrada que separe as duas, e roda o código dela contra o correto com diff_comportamental. No primeiro ponto de divergência, descobre qual hipótese é a verdadeira.
- 5. Redação e verificação.** O Treinador escreve uma dica ancorada nos valores reais (ex.: “observe o valor de soma quando i chega a 5”). Antes de enviar, o Verificador confere: está apoiada numa execução? não entrega a resposta? Aprovado → chega para a Marina.
- 6. Aprendizado do sistema.** Marina conserta de verdade e explica o erro em uma frase. Um observador dentro do Treinador atualiza o Modelo dela (subiu o domínio em “condição de parada”; ela usou a tática de ler variáveis). O Painel registra a decisão tomada e a evidência que a motivou.

O que cada seta carrega (os “envelopes”)

Envelope é só o pacote de dados que vai de uma peça para outra. Combinar esses envelopes é o que permite trabalhar em paralelo (ver seção 7).

De → Para	O que trafega
Treinador → Banco de Bugs	Pedido: { equívoco alvo, dificuldade }. Resposta: um item de bug completo (código, referência, testes, rótulo).
Treinador → RAG / Trajetórias	Pedido: { equívoco, perfil do aluno }. Resposta: caminhos de conserto que funcionaram com perfis parecidos.
Treinador → Ferramentas (MCP)	Chamada: { código, entrada }. Resposta: { saída, erro, tempo } — ou a lista de passos do trace.
Sub-rotina → diff_comportamental	Pedido: { código do aluno, código correto, entrada }. Resposta: { primeiro ponto de divergência, valores nesse ponto }.

De → Para	O que trafega
Treinador → Verificador	O envelope de mensagem: { texto, âncoras de execução, nível da dica, revela solução? }.
Verificador → Interface	{ aprovado / reprovado, mensagem final } — só o aprovado chega ao aluno.
Observador → Modelo do aluno	{ evidências verificadas por execução } → atualiza o domínio por equívoco e os sinais de depuração.

E na Fase A?

O fluxo é mais curto e é um laço: o Construtor entrega um bug gerado ao Validador; o Validador roda e responde “aprovado” ou “reprovado + motivo”; se reprovado, o Construtor tenta de novo com o motivo em mãos. Só bugs aprovados entram no Banco.

6. Divisão em módulos (4 frentes)

Quatro frentes que rodam em paralelo e se encaixam no fim. Abaixo, o perfil ideal de cada uma; vocês mapeiam os nomes.

Módulo	O que constrói	Perfil ideal	Cobre no edital
M1 — Motor de execução + ferramentas (MCP)	O sandbox e as ferramentas: executar, trace, rodar_suite, diff_comportamental, gerar_entrada_discriminante. Tudo exposto como MCP. É a fundação de que todos dependem.	Back-end / sistemas; confortável com Python “por baixo”.	Function Calling; Orquestração (ferramentas).
M2 — Fase A: banco de bugs	Ingestão do Refactory, rotulagem com ProgMiscon, o laço Construtor→Validador, a mineração de trajetórias (base do RAG) e a calibração de dificuldade.	LLM / agentes + manipulação de dados.	Multiagentes; base do RAG; ativo próprio.
M3 — Fase B: treinador + modelo do aluno	O agente Treinador, o Modelo do aluno, a política de decisão (por regras no MVP), o RAG em uso, a sub-rotina investigativa e o Verificador. É o cérebro e o diferencial.	O mais forte em agentes / LLM, com sensibilidade pedagógica.	Multiagentes; Memória e Privacidade; RAG; Avaliação e Segurança.
M4 — Experiência + medição + pitch	A interface da sessão (rodar entradas, ver variáveis), o painel de observabilidade (cada decisão + evidência), o painel do professor (versão leve), as métricas (taxa de bugs válidos, custo por sessão, ganho do agente sobre uma sequência fixa via ablação), o BMC, o pitch e os relatórios quinzenais.	Front-end / demo + pesquisa / comunicação.	LLMOps / observabilidade; Avaliação; Pitch; Equipe.

Caminho crítico: M1 (ferramentas) → M3 (o treinador que as usa) é a espinha da demo. M2 e M4 sustentam a narrativa (ativo próprio + medição). Se algo precisar encolher no MVP, encolhe em M2 e M4, nunca em M1 e M3. Como M1 é dependência de todos, ele entrega primeiro o formato das ferramentas (mesmo vazias) para não travar ninguém.

7. Contratos de interface

O que é um “contrato”: é o formato de dado combinado entre duas peças — o que uma entrega e o que a outra espera receber. Se todos concordam com esses formatos no Dia 0, cada um constrói a sua parte sozinho, usando uma versão “de mentira” (mock) da peça dos outros, e no fim tudo encaixa sem retrabalho. Os formatos abaixo são propostas iniciais — a ideia é discutir e travar juntos.

Contrato A — Ferramentas de execução (o que o M1 entrega)

Cada ferramenta tem uma entrada e uma saída fixas. Exemplo em formato de leitura fácil:

```
executar(codigo, entrada)
  -> { saida: texto, erro: texto|nulo, tempo_ms: numero }

trace(codigo, entrada)
  -> [ { linha: n, variaveis: {nome: valor, ...} }, ... ]

rodar_suite(codigo, testes)
  -> [ { teste_id, passou: sim/nao, esperado, obtido }, ... ]

diff_comportamental(codigo_aluno, codigo_ref, entrada)
  -> { divergiu: sim/nao, primeira_linha: n,
      variaveis_aluno: {...}, variaveis_ref: {...} }

gerar_entrada_discriminante(codigo_aluno, codigo_ref, hipoteses)
  -> { entrada: valor, separa: sim/nao }
```

Contrato B — Item do Banco de Bugs (o que o M2 entrega)

```
{
  id: texto,
  programa_bugado: codigo,
  referencia: codigo,           // versão correta
  testes: [ {entrada, saida_esperada}, ... ],
  id_equivoco: texto,           // ex.: 'IfIsLoop' (ProgMiscon)
  dificuldade: 1..5,
  origem: texto                 // 'refactory' | 'gerado'
}
```

Contrato C — Registro de Trajetória (base do RAG, do M2)

```
{
  id_equivoco: texto,
  estado_errado: resumo do código/valores antes,
  mudanca: o que foi alterado para consertar,
  estado_certo: resumo depois,
  fonte: 'dataset' | 'uso_real'
}
```

Contrato D — Modelo do Aluno (mantido pelo M3)

```
{
  aluno_id: texto (pseudônimo, sem dado pessoal),
  dominio_por_equivoco: { 'IfIsLoop': 0.0..1.0, ... },
```

```
habilidades_depuracao: {  
  forma_hipotese: 0..1,  
  testa_com_entrada: 0..1,  
  le_variaveis: 0..1 },  
historico: [ {bug_id, resultado, quando}, ... ]  
}
```

Contrato E — Envelope de Mensagem (o que o Verificador checa)

Nenhuma mensagem chega ao aluno sem passar por aqui. O Treinador entrega o envelope; o Verificador aprova ou barra.

```
{  
  texto: a mensagem para o aluno,  
  ancoras_execucao: [ referências à execução que sustentam  
                     cada afirmação do texto ],  
  nivel_dica: 1..4, // 1=pergunta socrática ... 4=nomear o equívoco  
  revela_solucão: sim/nao // Verificador confirma por execução  
}
```

Regra do Verificador: barra se ancoras_execucao estiver vazio (afirmou sem provar) OU se aplicar a mensagem permitiria copiar-e-passar. Só o envelope aprovado vira mensagem na tela.

8. O que entra no MVP de sábado

O hackathon regional é dia 26/09. A meta não é o sistema inteiro; é a fatia que prova a ideia ao vivo.

Entra (o mínimo que faz a demo brilhar)

- Ferramentas de execução funcionando: executar, trace, rodar_suite, diff_comportamental (M1).
- Alguns bugs prontos do Refactory, rotulados à mão com 5–10 equívocos de Python; e uma fatia pequena do laço Construtor→Validador rodando ao vivo (M2).
- O Treinador decidindo com regras + a sub-rotina investigativa + o Verificador (M3).
- A interface da sessão e o painel que explica cada decisão; uma métrica simples de ablação (M4).

A cena que precisa funcionar

Um código que compila, roda, não trava e falha silenciosamente num caso de teste. O juiz só diz “errado”. O Arandu levanta duas hipóteses, fabrica uma entrada que as separa, roda instrumentado, descarta uma e explica o equívoco mostrando os valores reais — sem nunca entregar a correção. Se isso for demonstrado, o critério técnico (30% da nota) está bem defendido.

Fica para a fase final

- Base própria a partir de um recorte anonimizado do CodeBench (dados de longo prazo, ligação com evasão).
- RAG alimentado pelo uso real (o flywheel) e política de decisão aprendida.
- Painel do professor completo e piloto com turma real.

9. Para quem quiser se aprofundar

Poucos ponteiros, caso alguém queira ir além (não é leitura obrigatória):

- [LDB — depurar verificando a execução passo a passo \(base do laço investigativo\)](#)
- [Teaching LLMs to Self-Debug \(raciocinar sobre o resultado da execução\)](#)
- [ProgMiscon — catálogo de equívocos \(inclui Python\)](#)
- [Refactory — a base de código de alunos do protótipo](#)
- [A Survey of Knowledge Tracing \(modelo do aluno / BKT\)](#)